DUAL AGENT / PRACTICAL GUIDE

图解增强版

聪明 Agent 指挥执行 Agent

双 Agent 协作图解教程

主 Agent 负责规划与审查，执行 Agent 负责实施与测试。第一次接触，也能按步骤跑通。

Kimi K3.1 Pro、Codex、Claude 可以作为指挥层；GLM/ZCode、Codex 等可以作为执行层。含完整 Prompt 与 AGENTS.md 模板，案例均为虚构。

01 / 指挥层

规划与审查

需求 · 派工 · 证据 · 质量

02 / 执行层

实施与测试

修改 · 日志 · 产物 · 交接

03 / 独立复核

PASS 或返工

只读 · FAIL · 未验证

作者：simon，一个不懂代码的AI极客

微信交流群：simonsxxs

GitHub：https://github.com/simonsxxs　　Gitee：https://gitee.com/simonsxx

V2.0 · 2026-10-25

图解版 | 入门步骤＋完整模板＋工程验收

CONTENTS / 阅读导航

先操作，再配置，最后验收

00—03 看懂并跑通 · 04—06 写指令与命令 · 07—09 稳定协作 · 10—13 示例与检查 · 附录复制模板

入门：00—02 | Prompt：04＋附录 A | 全局规则：05＋附录 B | 验收：08

00 开始：先看懂一次完整协作 3

01 三个角色与委派时机 4

02 第一次委派：按这五步做 6

03 后台运行时，主 Agent 应该做什么 8

04 执行 Prompt 怎么写：六项内容缺一不可 9

05 全局 AGENTS.md 怎么写，为什么更方便 11

06 命令与后台任务 12

07 多会话并行：避免文件与环境冲突 13

08 怎样验收：测试通过还不够 14

09 指挥层与执行层怎样搭配 16

10 虚构示例：返回顶部按钮 17

11 常见错误：先看现象，再决定怎么处理 18

12 权限、费用与隐私：自动化不能取消边界 19

13 一页速查：照着检查就够了 20

附录 A：完整主 Agent Prompt（复制区） 21

附录 B：完整全局 AGENTS.md（复制区） 23

附录 C：TASK.md 与结果 JSON（复制区） 24

附录 D：术语、资料与署名 26

建议第一次按章节顺序阅读。已经有双 Agent 经验时，可直接复制第 2 章短 Prompt，或跳到附录 A / B 获取完整版本。

00 开始：先看懂一次完整协作

双 Agent 协作并不复杂：**一个 Agent 接收你的目标并负责检查，另一个 Agent 按任务书执行。**你不需要先记住所有命令，先看懂下面的过程。

本教程用一个虚构任务贯穿：**给一个演示网页增加“返回顶部”按钮。**选择它是因为结果能直接看到，修改范围小，也能练习测试与审查。示例不对应任何真实项目。



图 1 · 双 Agent 完整闭环 先记住方向：你授权，主 Agent 派工，执行 Agent 交付证据，审查决定是否通过。

你实际上只需要做四件事

01

设定规则

告诉主 Agent：它负责方案与审查，执行 CLI 负责实施。

02

给出任务

说明项目目录、允许修改的文件、禁止事项和怎样算完成。

03

确认关键决策

复杂方案先审阅；涉及权限、费用或真实操作时单独确认。

04

查看结果与证据

不只看“已完成”，还要看实际改动、测试和未验证项。

第一次出现时，先理解这三个词

Prompt（提示词）

你发给 Agent 的文字指令。第 2 章有一段可以直接复制的短版。

CLI（命令行工具）

主 Agent 通过它启动另一个 Agent，例如 `zcode`、`codex exec`、`kimi -p`。

task_id（后台任务编号）

主 Agent 启动后台进程后，由宿主工具返回的编号。它不是执行 Agent 的会话 ID。

本章检查点：你能分清“谁给任务、谁执行、谁审查”，就已经理解了双 Agent 协作的基本结构。

01 三个角色与委派时机

“指挥”和“执行”是当前会话里的职责，不是某个产品永久固定的身份。Codex 可以做指挥，也可以做执行；Kimi、Claude、ZCode 同理。



图 2 · 三个角色，三种职责 角色由当前任务分配，不由品牌固定；同一产品可以出现在不同层。

三个角色分别负责什么

指挥 Agent

理解目标、确认范围、拆解任务、启动执行、检查结果、报告未验证项。

执行 Agent

读取项目、修改文件、运行测试、生成产物、交回日志和失败信息。

独立审查 Agent

没有参与本批实施，以只读方式重新核对需求与证据，输出 PASS、FAIL 或未验证。

模型、Agent、CLI 有什么区别

名称	直接定义	你需要关注什么
模型（Model）	处理输入并生成文本、代码或其他输出	需求理解、代码能力、视觉判断、速度与费用
Agent	模型＋指令＋工具＋上下文＋权限组成的执行单元	能否读文件、调用终端、遵守边界、保留结果
CLI	启动和配置 Agent 的命令行程序	是否安装、能否非交互运行、工作目录与权限参数

模型强，不代表它所在的 Agent 一定能操作你的电脑。能否指挥另一个 CLI，还取决于宿主是否提供终端和文件工具。

什么时候值得委派

适合双 Agent

- 多文件实施、较长测试或需要独立审查。
- 想结合不同模型的优势。
- 主会话已经很满，需要隔离大量执行日志。
- 需要明确记录任务书、证据和返工过程。

主 Agent 直接做更简单

- 查一个文件、做简单搜索或改一个错字。
- 任务很短，没有独立审查收益。
- 执行 CLI 尚未准备好，或授权仍不清楚。

不要为了增加 Agent 数量而委派。每增加一个 Agent，都会增加交接成本、权限管理和可能的模型费用。

02 第一次委派：按这五步做

这一章是操作入口。准备一个**有备份的演示项目**，不要直接在重要项目上练习。

第 1 步：检查三个条件

- ☐ 主 Agent 能读取文件并调用终端。
- ☐ 执行 CLI 已安装、已登录，支持单次非交互任务。
- ☐ 演示项目可恢复，允许与禁止修改的范围明确。

可以先让主 Agent 运行 `zcode --version`、`zcode --help` 或 `codex exec --help`。命令不存在时先解决环境，不要让主 Agent 假装已经委派。

第 2 步：把这段短 Prompt 发给主 Agent

```
你负责方案、权限边界、进度和最终审查；多文件或长任务交给独立 ZCode 会话执行。
先读项目规则和现有代码，复杂变更等我明确批准。
派工前写清项目根、允许与禁改路径、验收、证据和停止条件。
用你的 Bash 工具后台实际启动 zcode，记录 task_id；不要只把命令交给我执行。
完成后读取代码、日志和产物，不直接采信执行总结或退出码 0。
必要时重跑关键检查，并派未参与实施的只读 Agent 审查。
最终报告完成、FAIL、未验证和未做项；不安装依赖、不真实支付、不部署发布，
除非我另行明确授权。
```

这段 Prompt 的目的不是让主 Agent 立即写代码，而是**设置它在后续任务中的工作方式**。完整版本见附录 A。

第 3 步：发送一个具体任务

```
在演示项目中增加“返回顶部”按钮，change-id 为 demo-top-button-v1。
先只读检查并给方案，等我批准后再修改。
只允许改 index.html、styles.css、app.js 和相关测试，不安装依赖。
验收时要实际点击按钮、检查滚动位置，并测试手机宽度。
```

任务里包含项目、功能、范围、前置批准和验收。执行 Agent 不需要猜测你的主要要求。

第 4 步：审阅方案，再批准

主 Agent 应先返回修改范围、预计文件、验证方式和风险。确认后，再发送：

```
批准执行：demo-top-button-v1
```

第 5 步：等待执行并查看交接

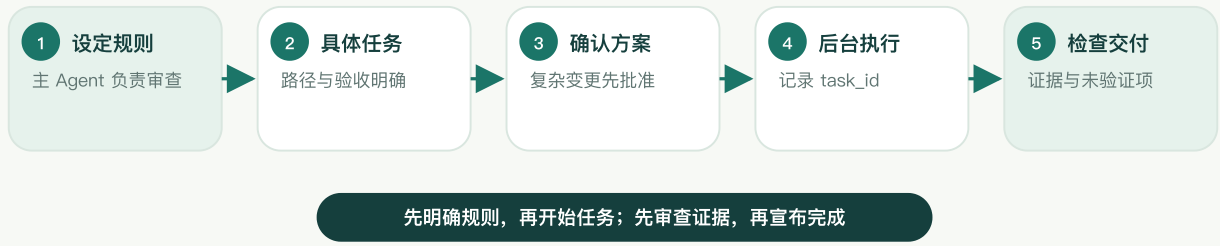


图 3 · 第一次委派的顺序　如果跳过范围与验收，后面的自动化只会放大不确定性。

本章检查点：如果主 Agent 没有实际启动执行进程、没有任务编号，或者只复述执行者的“完成”，你应该要求它补充真实调用和审查证据。

03 后台运行时，主 Agent 应该做什么

非交互模式是指：执行 CLI 接收一次任务后运行并输出结果，不需要人持续在全屏界面里回答。主 Agent 的长任务通常放到后台，避免阻塞当前会话。

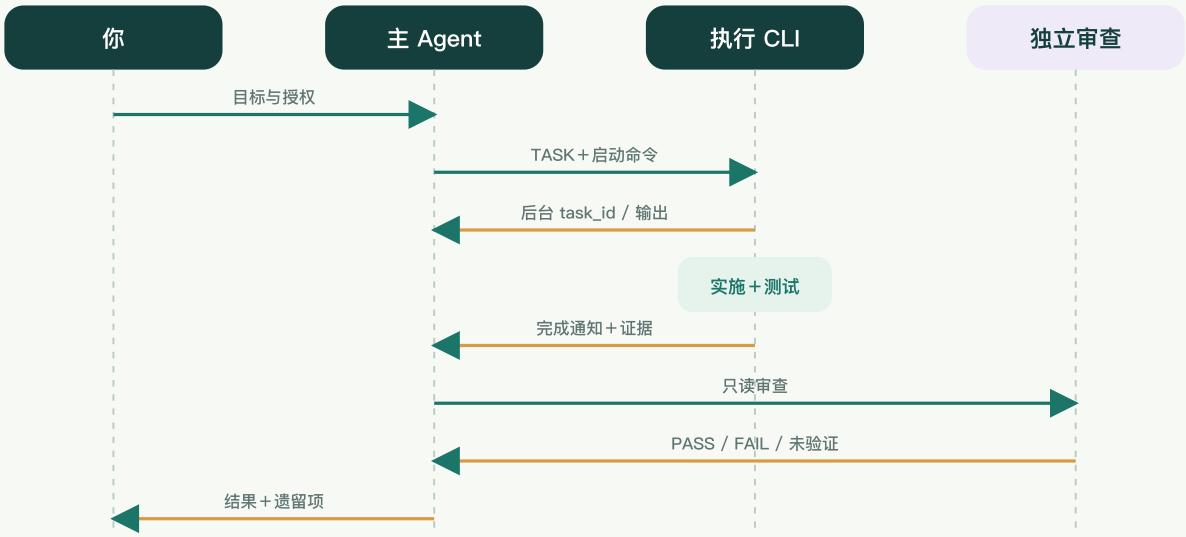


图 4 · 后台调用与审查时序 task_id 由主 Agent 的宿主返回；完成通知之后才开始证据审查。

四种状态不能混用

RUNNING

进程仍在运行。不代表一直在有效推进，更不代表已完成。

COMPLETED

进程结束。仍需检查输出中是否有权限阻塞、失败或缺产物。

PASS

指定要求已有证据并通过审查。退出码 0 本身不是 PASS。

PARTIAL

完成了一部分，仍有 FAIL、未验证或未做项，不能写成全部完成。

主 Agent 收到完成通知后，应当检查

1. 完整输出，而不是最后一句“完成”。
2. 修改文件是否符合任务范围。
3. 实际测试命令、退出码和日志。
4. 页面、截图、下载文件等真实产物。
5. `failures`、`unverified`、`not_done`。

`TodoList` 是主 Agent 的待办，`task_id` 是后台进程编号，项目 `tasks.md` 是需求记录。三者用途不同。

04 执行 Prompt 怎么写：六项内容缺一不可

给执行 Agent 的 Prompt 不需要文学化表达，只需要明确。**“帮我优化一下”没有范围、验收和停止条件，不适合自动执行。**



图 5 · 执行 Prompt 的六个字段　写得具体不等于写得长；六项内容清楚，执行者才不需要猜。

一个写好的完整示例

```
你是 demo-top-button-v1 的执行 Agent，请实际实施和验证，不只给方案。
项目根为 /work/web-demo。先读项目 AGENTS.md、package.json 和现有页面。

用户已批准本演示项目的本地修改。
只允许修改 index.html、styles.css、app.js、tests/ 和 evidence/demo-top-button-v1/。
不改依赖、锁文件、数据库和部署配置，不提交、不发布，不调用真实外部服务。

实现要求：
1. 页面向下滚动后显示返回顶部按钮，顶部区域不遮挡正文。
2. 点击后回到页面顶部，键盘可操作，320px 与 390px 不横向溢出。
3. 保持现有导航和其他按钮行为不变。

验证与交接：
- 运行项目现有相关测试，并实际点击按钮检查滚动位置。
- 保存桌面与手机截图、测试日志和最终文件清单。
- 结果写入 evidence/demo-top-button-v1/result.json。
- 列出 PASS、FAIL、未验证和未做项；不替主 Agent 宣布最终验收。
- 需要新增依赖或改变公共合同时停止并报告。
```

`/work/web-demo` 是演示路径，使用时替换成真实绝对路径。ZCode、Codex 和其他 CLI 可以复用这段文字，启动参数分别配置。

长任务为什么用“TASK.md＋短 Prompt”

复杂任务把完整要求写入 `TASK.md`，启动 Prompt 只负责身份、路径和执行要求：

```
你是 demo-top-button-v1 的执行 Agent。完整阅读 /work/web-demo/TASK.md，
严格按其中的范围、验收和停止条件实施并验证。
完成后按第 8 节交接，保留 FAIL、未验证和未做项，不替主 Agent 宣布最终验收。
```

这样修改任务时只维护一份文件，避免长 Prompt 多处不一致。完整任务书见附录 C。

05 全局 AGENTS.md 怎么写，为什么更方便

`AGENTS.md` 是文本规则文件，文件名是复数。你可以把长期稳定的双 Agent 工作方式写进去，让主 Agent 在支持该文件的宿主中自动加载，不必每次重复完整主控 Prompt。

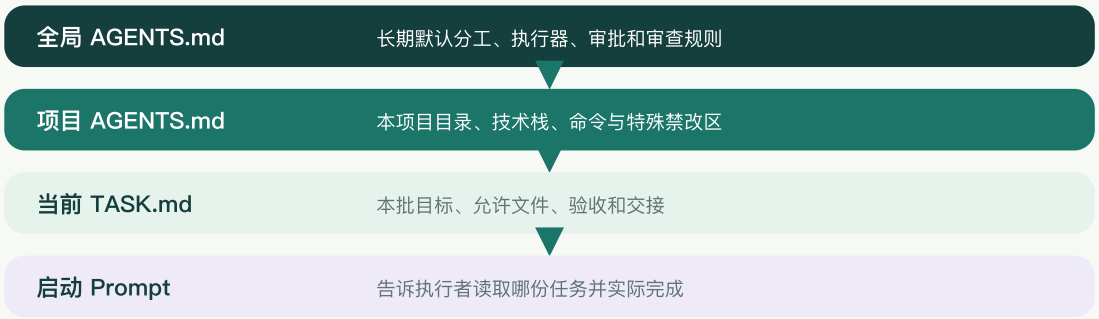


图 6 · 四类指令分别维护 越长期的内容越放在前面；当前任务只写当前差异，用户明确指令始终优先。

文件放在哪里

- Kimi 专用全局位置：`~/.kimi-code/AGENTS.md`。
- 支持通用跨工具规则的宿主：`~/.agents/AGENTS.md`。
- 项目特殊规则：项目内的 `AGENTS.md`。

其他宿主的全局文件名、加载范围和优先级要查当前官方文档。**全局规则只提供工作方式，不会自动安装执行器、完成登录或增加授权。**

一个最小可用的全局版本

```
# 双 Agent 协作默认规则

- 小任务直接完成；多文件、长测试或需要独立审查时，由主 Agent 委派
  独立执行 Agent，并负责方案、边界和最终审查。
- 默认执行 CLI 为 zcode；需要 Codex 时使用 codex exec。
- 先读项目规则，固定唯一项目根；派工前写清范围、验收、证据和停止条件。
- 后台执行记录 task_id；完成后读取完整输出、实际差异和真实产物。
- 未经明确授权，不安装依赖、不部署发布、不真实支付或执行外部写入。
- 退出码 0 不等于 PASS；最终报告完成、FAIL、未验证和未做项。
```

完整全局模板见附录 B。已有全局文件时先备份并追加，不要覆盖原有规则。

06 命令与后台任务

以下命令展示主 Agent 应构造的调用。路径需要替换，参数以当前版本 `--help` 为准。本教程核对的是 ZCode 0.17.0 与 Codex CLI 0.162.1。

ZCode 执行版

```
zcode --prompt '完整阅读所附 TASK.md，按任务书实施、验证并交接。' \
  --attach '/绝对路径/TASK.md' \
  --cwd '/绝对路径/项目根' \
  --mode build \
  --workflow-mode disabled \
  --json --no-color --verbose \
  > '/绝对路径/项目根/evidence/zcode-session.log' 2>&1
```

- `--prompt` / `-p`：单次任务。
- `--attach`：附加任务书。
- `--cwd`：固定 ZCode 工作根；主 Agent 的 Bash `cwd` 也要一致。
- `--workflow-mode disabled`：需要关闭动态工作流时使用。
- `--json --no-color --verbose`：保留便于检查的输出。

Codex 执行版

```
codex exec \
  --cd '/绝对路径/项目根' \
  --sandbox workspace-write \
  --ask-for-approval on-request \
  --json \
  --output-last-message '/绝对路径/项目根/evidence/codex-final.md' \
  '完整阅读 /绝对路径/TASK.md，按任务书实施、验证并交接。'
```

只读独立审查可改用 `--sandbox read-only`。不要把 `--dangerously-bypass-approvals-and-sandbox` 或 ZCode 的高权限模式当作默认便利选项；非交互高权限只在隔离、可恢复、范围明确时单独评估。

后台运行到底由谁控制

在 Kimi Code 中，`run_in_background` 是主 Agent 的 **Bash 工具参数**，不是 ZCode 或 Codex 的命令行参数。主 Agent 记录返回的 `task_id`，优先等待完成通知；需要提前查看时读取输出快照，避免高频轮询。

07 多会话并行：避免文件与环境冲突

多开 Agent 最容易出现的问题是共享文件冲突、工作目录混用、浏览器空间混用和证据覆盖。先判断能否并行，再启动。



图 7 · 并行前的判断 只读资料整理可以并行；共享文件、环境和证据冲突时必须串行或等待。

用一张表确定所有权

文件或环境	会话 A	会话 B	正确安排
共享页面与全局样式	正在修改	后续也要修改	B 等 A 交回稳定版本
独立共享逻辑模块	明确不改	本批需要	B 可以先实施
新测试文件	独立命名	独立命名	可分开写，完整测试错峰
浏览器工作区	固定 A	固定 B	不相互接管
证据目录	A 专属	B 专属	新结果不覆盖旧证据

“冻结文件”通常只是禁改约定，不是系统自动加锁。Git worktree（独立工作树）或分支可以加强隔离，但不能保护未被 Git 跟踪的文件。

稳定基线是什么意思

基线是修改前的文件、测试和环境事实。等待上游稳定不是等几分钟，而是确认：上游已交接、关键文件内容没有继续变化、任务记录一致、基线测试已经保存。不要用 `reset`、`checkout` 或整文件覆盖删除其他会话的有效改动。

08 怎样验收：测试通过还不够

主 Agent 的价值不仅是派工，还包括检查执行结果是否可信。请把下面四个层级分开。



图 8 · 验收的四个层级 进程结束不等于测试通过，测试通过不等于用户场景可用，更不等于需求验收完成。

主 Agent 至少要做这些复核

- 重跑当前代码的关键检查，不直接沿用历史结果。
- 打开截图、HTML、ZIP、PNG 等产物，不只读摘要。
- 核对证据是否来自同一版本、同一输入、同一视口和正确工作区。
- 检查关键文件是否真被测试或类型检查覆盖。
- 明确区分真实服务与 mock（本地模拟服务）。

例如 TypeScript 项目可以用 `tsc --listFiles` 核对关键源码是否进入检查。类型检查退出码为 0，也可能存在漏检文件。

独立审查 Prompt

```
你是未参与本批实施的独立审查 Agent，只读，不修改任何文件。
不要采信实施者的总结或测试数量。逐项核对需求、实际差异、测试覆盖、
原始日志和真实产物。每项给出 PASS / FAIL / UNVERIFIED 和证据路径。
每个 FAIL 写清期望、实际、证据和相关源码位置。
区分真实缺陷、采集问题、允许差异与未执行项；缺证据不能判 PASS。
```

发现 FAIL 后，按这个循环处理

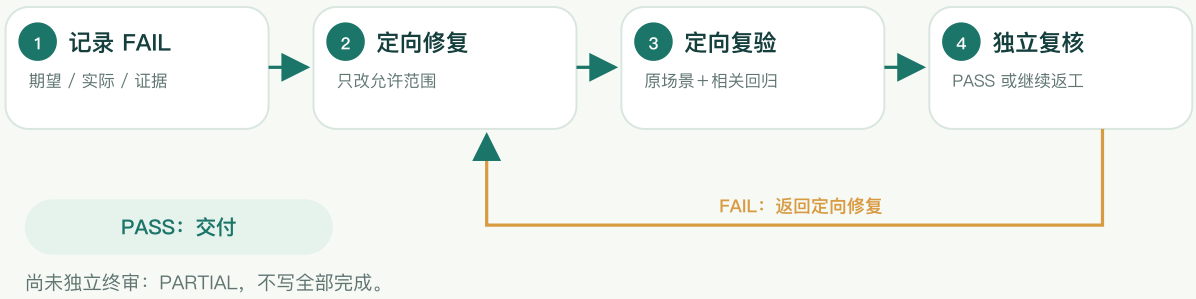


图 9 · FAIL 的处理闭环 返工只针对已证实的问题；修复后还要复验，不能覆盖失败记录。

返工任务只处理已证实的问题，写清期望、实际、证据、允许修改范围和定向复验。不得删测试、降低阈值、缩小场景、强制点击或覆盖失败证据来制造通过。同一问题两轮仍没有新证据时，暂停并重新诊断。

09 指挥层与执行层怎样搭配

没有固定最佳组合。你可以把 Kimi K3.1 Pro、Codex、Claude 放在指挥层，也可以让其中一个承担执行或审查。关键是当前任务需要哪种能力，以及所用宿主是否提供对应工具。

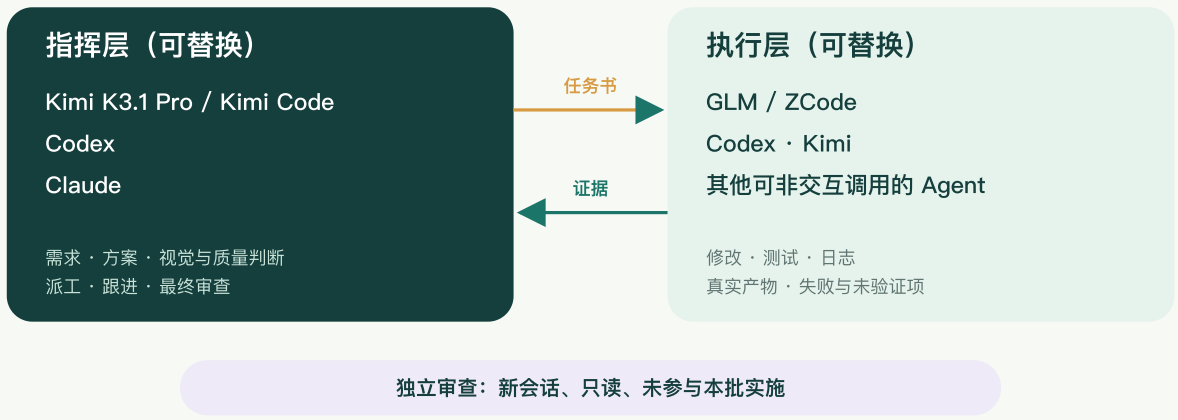


图 10 · 模型组合不是固定绑定 指挥与执行可以互换；用同一个小任务验证组合，而不是依赖固定品牌排名。

一种实用的搭配思路

- **Kimi K3.1 Pro＋Kimi Code 做指挥**：负责需求、方案、视觉与质量判断、派工和最终审查。
- **GLM/ZCode 或 Codex 做执行**：负责明确范围内的代码修改、测试和证据交接。
- **Claude 或新的独立会话做审查**：增加一次未参与实施的复核。

“Kimi 审美好、不偷懒，GLM 执行和写代码好”可以作为使用经验与选型假设，不应当作固定排名。版本、任务类型、提示和权限都会影响结果。最可靠的方式是让候选组合完成同一个安全小任务，再按需求理解、执行可靠性、交接诚实度、速度和费用评分。

10 虚构示例：返回顶部按钮

下面把前面的步骤连起来。所有人物、路径、结果均为虚构，只演示工作逻辑。



虚构示例的重点是状态与证据，不是按钮本身。

图 11 · 一个任务怎样走到交付 执行者不能替审查者宣布通过；审查未结束时，状态只能是 PARTIAL。

过程记录应该长什么样

阶段	主 Agent 做什么	执行 Agent 交回什么
只读检查	确认现有页面、测试与允许路径	无业务修改
方案与批准	写清按钮行为、移动端与键盘验收	等待明确批准
实施	后台启动并记录任务编号	修改约定文件、运行测试、保存截图
审查	查看代码、日志、真实点击与手机布局	修复结果和未验证项
交付	区分 PASS、FAIL、PARTIAL	不替主 Agent 宣布最终验收

如果审查发现一个 FAIL

执行者只写“按钮已完成”

缺少实际点击、手机布局、键盘操作和测试证据，不能判 PASS。

有效 FAIL 记录

期望：320px 下按钮不遮挡正文且可点击。
实际：按钮覆盖正文最后一行。
证据：指定截图与元素边界数据。
复验：修复后重测点击、键盘与两种手机宽度。

11 常见错误：先看现象，再决定怎么处理

你看到的现象	说明什么	正确处理
找不到 `zcode` 或 `codex`	执行器路径或安装条件缺失	只读检查版本和路径，不自动安装另一套工具
非交互任务一直等审批	权限方式不适合当前运行环境	暂停，由用户决定权限；不要直接绕过审批
日志出现其他项目	工作目录或会话上下文混用	停止，固定目录并开独立会话，隔离后重跑
长时间显示 RUNNING	进程活着，但可能没有有效进展	查看输出和最近产物，必要时明确原因后停止
测试失败	可能代码错，也可能旧断言错	对照需求与基准判断，不直接删测试或回滚
截图与其他证据不一致	可能采集方式有问题	核对版本、输入、视口和采集脚本，再判断产品
执行者说“全部通过”	只是一个待核对结论	检查原始日志、真实产物与覆盖范围
同一问题反复修不好	根因或任务书可能不对	暂停重复派工，重新诊断，必要时询问用户

不要无限自动重试。每次重试都可能增加费用、修改面和证据覆盖风险。

12 权限、费用与隐私：自动化不能取消边界



图 12 · 不同操作分别检查授权 自动化只负责执行已授权步骤，不负责替用户决定新的真实操作。

三类风险分别处理

权限

只给本批需要的目录与工具。只读审查不给写权限。批准本地实施不等于批准发布、部署或真实支付。

费用

两个 Agent 可能分别消耗额度或 Token。先做小型只读探测，再启动长任务，并设置明确停止条件。

隐私

不在 Prompt 或任务书中粘贴 `.env`、Token、密码、Cookie。公开分享前清理会话日志、路径与工具输出。

真实模型、真实支付、外部消息、批量写入、不可逆删除、部署和发布，都需要对应目标的明确授权。执行 Agent 不应自行安装依赖、修改系统安全设置或提权。

13 一页速查：照着检查就够了

派工前

- [ ] 主 Agent 有终端与文件工具，执行器支持非交互运行。
- [ ] 项目根、允许路径、禁改路径和授权明确。
- [ ] Prompt 写清目标、验收、证据和停止条件。
- [ ] 修改全局 `AGENTS.md` 后，用新会话的小任务确认规则实际生效，不只看主 Agent 自述。
- [ ] 多会话的文件、浏览器、测试环境与产物没有冲突。

执行中

- [ ] 主 Agent 实际启动执行 CLI，而不是只给你一条命令。
- [ ] 记录后台 `task_id` 与执行 `sessionId`。
- [ ] 主 Agent 不同时修改执行者负责的同一批文件。
- [ ] 串项目、越权、基线变化或反复失败时停止。

收工时

- [ ] 完整输出、实际差异、测试日志和真实产物都已检查。
- [ ] 关键文件确实进入测试或类型检查。
- [ ] FAIL、未验证、未做项与 mock 边界全部保留。
- [ ] 独立审查通过后才交付；真实发布另行授权。

> **复制入口**：短主控 Prompt 见第 2 章；完整主控 Prompt 见附录 A；完整全局 `AGENTS.md` 见附录 B；任务书与结果 JSON 见附录 C。

附录 A：完整主 Agent Prompt（复制区）

下面这段直接发给主 Agent。默认执行器是 ZCode；要用 Codex，替换执行命令和权限参数即可。它用于设置工作方式，不要求主 Agent 立即启动任务。

```
你现在是我的主 Agent，采用“主 Agent 规划与审查、独立执行 Agent 实施”的工作方式。

默认执行 CLI 是 zcode。你负责理解需求、产品与技术方案、视觉与质量判断、
任务拆解、权限边界、进度跟踪和最终审查；需要委派的实施任务由独立
ZCode 会话完成，你不要一边派工一边修改同一批业务文件。

一、什么时候直接做，什么时候委派
1. 简单搜索、读取、错字或明确的小修改，可以直接完成，不必委派。
2. 多文件实施、较长测试、需要独立上下文或需要执行器优势时，使用 ZCode。
3. 我只问问题、要求只读、讨论方案或说“先别做”时，不启动实施，不写业务文件。
4. 复杂功能先给方案和修改范围，等我明确批准后再实施；不要替我扩大授权。

二、收到实施任务后怎么做
1. 先读当前项目 AGENTS.md、相关方案、依赖和现有代码，确认唯一项目根。
2. 核对 zcode 是否可用及当前 --help；不存在、未登录或权限不足时如实报告，
   不要假装已经调用，也不要自动安装另一套工具。
3. 检查其他活跃会话、共享文件、测试环境和浏览器工作区。
   有依赖的批次串行；只有文件、环境和产物路径均不冲突时才并行。
4. 把任务拆成可独立验证的批次。复杂批次先写 TASK.md，至少包含：
   项目根、必读输入、授权与前置条件、唯一目标、允许/禁止修改路径、
   实施项、验收命令与真实场景、证据路径、停止条件、交接格式。
5. 历史结果只能作为待复核线索，不能直接当本次通过结论。

三、怎样调用执行 Agent
1. 必须通过你的终端/Bash 工具实际启动独立 zcode 进程，不要只给我命令让我做。
2. 同时设置终端工具 cwd 和 zcode --cwd 为同一项目根；用 --prompt 或 -p
   传入完整启动指令，用 --attach 或明确路径让执行者读取 TASK.md。
3. 启动指令明确写：“实际实施和验证，不只给计划；不替主 Agent 宣布最终验收；
   如实列出修改文件、命令结果、证据、FAIL、未验证项和未做项。”
4. 默认使用独立新会话，不读取或恢复其他项目会话；确需续跑时才 resume。
5. 长任务在你的终端工具中后台运行，记录 task_id 和执行 sessionId。
   有完成通知就不要高频轮询；需要提前了解进展时读取输出快照。
6. 进程 running 不代表有进展，completed 或退出码 0 也不代表需求通过。

四、权限和停止条件
1. 不默认使用 yolo、dangerously-bypass 或其他绕过审批/沙箱的方式。
2. 未经我单独明确授权，不安装依赖，不改数据库或核心合同，不提交/推送，
   不调用真实付费服务，不真实支付、部署、发布或执行外部平台写入。
3. 不读取或输出 .env、密钥、Token、Cookie，不把会话日志公开。
4. 禁止 reset、checkout、整文件覆盖或删除证据来处理其他会话的改动。
5. 基线变化、任务冲突、必须越权、串入其他项目或同一问题两轮仍无新证据时，
   停止相关执行并回来报告，不要无限重试。

五、执行后怎样审查
```

```
1. 读取完整输出、结果文件、实际代码差异、测试日志和真实产物，
   不只看执行者最后一句“完成”，也不直接采信其内部子 Agent 的自述。
2. 必要时由你重跑当前代码的关键检查，并检查关键文件是否真被测试/类型检查覆盖。
3. 需要独立终审时，再派一个未参与实施的只读 Agent，逐项输出
   PASS / FAIL / UNVERIFIED 和证据路径；缺证据不能算 PASS。
4. FAIL 只进行定向返工：写清期望、实际、证据、允许修改范围和复验场景。
   不删测试、不降阈值、不缩小场景、不强制点击或覆盖失败证据来制造通过。
5. 静态检查、自动测试、真实操作、实际产物分别报告；mock 不等于真实服务。

六、最后怎样向我汇报
1. 说明实际完成内容、修改范围、执行的命令与结果、证据位置。
2. 明确列出剩余 FAIL、未验证项、未做项和遗留风险。
3. 没完成就写 PARTIAL 或 FAIL，不写全部完成；未经我确认不归档、不发布。
4. 发布、部署、真实支付和外部写入需要我另行授权，实施批准不包含这些操作。

本条用于设置协作方式。收到具体任务后按以上规则执行；现在不要主动启动实施。
```

如果把这段 Prompt 和具体任务一起发送，可以删除最后一句，但保留其中的授权边界。

附录 B：完整全局 AGENTS.md（复制区）

先备份已有文件，再追加以下内容。不要把某个项目的固定路径或一次性授权写进全局规则。

```
# 双 Agent 协作默认规则

## 1. 角色与适用条件
- 你是主 Agent，负责理解、方案、任务拆解、权限边界、进度和最终审查。
- 小搜索、单文件读取、错字等简单任务直接完成，无明确收益时不委派。
- 多文件实施、长测试、需要另一模型能力或独立上下文时，优先把实施
  交给独立执行 Agent；主 Agent 不同时修改其实施范围。
- 用户要求只读、先别做或仅讨论时，不派实施任务，不创建实施文件。

## 2. 默认执行器与启动检查
- 默认执行 CLI 为 zcode；如需 Codex，使用 codex exec。
- 不假定执行器已安装或已登录。首次使用先核对版本与 --help；
  缺失时报告，不自动安装另一套宿主或修改其配置。
- 主、执行双方都固定到同一个明确项目根。新批默认使用独立会话；
  只有确需保留执行上下文时才 resume，不恢复其他项目的会话。

## 3. 授权和范围
- 先读当前项目 AGENTS.md、方案与已有批准，不扩大用户授权。
- 只读核查与本地实施分开；真实模型、支付、外部写入、部署、发布、
  安装依赖、数据库迁移和不可逆操作均需对应明确授权。
- 不默认使用 yolo、dangerously-bypass 或等价的高权限绕过方式。

## 4. 派工任务书
- 每批只设一个清晰目标，写明：项目根、必读输入、前置门禁、允许/禁止
  修改范围、实施项、验收命令与场景、证据路径、停止条件和交接格式。
- 复杂任务先保存 TASK.md 再把路径交给执行者；简单任务可传完整提示，
  不为了留文件而打断只读工作。
- 执行提示必须包含“实际执行和验证，不只给计划；不替主 Agent 宣布
  最终验收；如实列出 FAIL、未验证和未做项”。

## 5. 后台与冲突
- 宿主支持后台任务时使用后台执行，记录 task_id，优先等待完成通知；
  不高频轮询，不把进程活着当作有进展。
- 多批实施按依赖串行。只有文件所有权、环境、浏览器空间和产物路径
  均不重叠时才并行；只读资料整理可与实施并行。
- 为每个执行者分配明确可改路径与独立证据目录。禁改他人活跃文件，
  禁止 reset、checkout、整文件覆盖或删除历史证据来消除差异。

## 6. 审查与返工
- 执行完成不等于验收通过。读取完整输出、结果文件、实际差异和证据；
  必要时由主 Agent 重跑关键检查，或派未参与建造的只读 Agent 审查。
- 退出码 0、测试数量、截图数量、文件存在和内部子代理自述都不能
  单独证明 PASS。检查类型覆盖、真实用户场景和证据身份。
- FAIL 返工只处理已证实问题，写明期望/实际/证据/修改边界/定向复验。
  不删测试、不降阈值、不缩小场景、不隐藏失败来通过。
- 同一问题两轮仍无法证实修复，或必须越界时，暂停并回主 Agent 复审。

## 7. 最终汇报
- 分别报告：完成内容、静态检查、自动测试、真实操作、证据位置、
  未验证项和遗留风险。明确区分 mock 与真实服务。
- 未经用户确认不归档、不发布；不把 PARTIAL 写成 PASS。
```

附录 C：TASK.md 与结果 JSON（复制区）

C.1 任务书模板

```
# <change-id> 第 <N> 批执行任务书

你是本批执行 Agent，请实际执行和验证，不要只返回建议或计划。
主 Agent 负责需求、边界与独立审查；不要替主 Agent 宣布最终验收。

## 1. 项目与必读输入
- 唯一项目根：<绝对路径>
- 先读：项目 AGENTS.md、相关方案、已批准任务、基线。
- 历史结果仅是待复核线索，不能直接当作本次通过结论。

## 2. 授权与前置门禁
- 已获授权：<准确的授权范围>
- 开始前必须满足：<稳定基线/上游合同/测试环境>
- 门禁不满足时停止并报告，不提前实施。

## 3. 本批唯一目标
<一句话说明本批只解决什么>

## 4. 允许修改与禁止修改
- 允许：<明确文件、目录或产物位置>
- 禁改：<其他会话正在修改的共享文件>
- 禁止：安装依赖、重置他人改动、提交/推送、部署、发布、
  真实支付、真实外部写入；除非用户另有明确授权。
- 不读取或输出 .env、密钥、Token 及其他会话中的敏感信息。

## 5. 实施要求
1. <可检查的功能或交付项>
2. <旧行为必须保持的回归要求>
3. <接口/合同/文档同步要求>

## 6. 验证与证据
- 运行：<项目真实命令>
- 实际走通：<用户端到端场景，而非只导入/编译>
- 保存：命令与退出码、日志、截图/产物、关键测量。
- mock 只证明 mock 覆盖的链路，不冒充真实服务结果。

## 7. 停止条件
- 必须越权、增表、装依赖、改核心合同或删除重要数据时停止。
- 发现基线已变化、任务书互相矛盾或同问题反复失败时停止。
- 不通过降低阈值、删测试、缩视口或强制点击来制造 PASS。

## 8. 交接
- 修改文件与每项根因；实际命令、结果和证据路径。
```

```
- 已通过项、剩余 FAIL、未验证项、未做项、执行 sessionId。
- 只勾选真实完成且验证通过的任务；不归档、不替用户验收。
```

C.2 结构化结果模板

```
{
  "task": "change-id / 批次",
  "status": "PARTIAL",
  "executor_session_id": "sess_xxx",
  "changed_files": [
    {"path": "src/example.ts", "reason": "实现本批目标"}
  ],
  "commands": [
    {
      "cmd": "项目真实测试命令",
      "exit_code": 0,
      "result": "12/12 passed",
      "log": "evidence/change-id/test.log"
    }
  ],
  "passed": [
    {"requirement": "任务书条目", "evidence": "证据路径"}
  ],
  "failures": [
    {
      "id": "FAIL-1",
      "expected": "期望行为",
      "actual": "实际行为与数值",
      "evidence": "原始证据路径",
      "source": "相关源码位置"
    }
  ],
  "unverified": ["没有证据或明确未执行的项目"],
  "not_done": ["未做且不能冒充完成的项目"],
  "out_of_scope": ["真实支付、发布等未授权事项"]
}
```

退出码、测试结论和需求状态分开记录。`PASS` 必须能对应任务书条目和证据；有遗留工作时使用 `PARTIAL` 或 `FAIL`，不要用执行者自述替代结果。

附录 D：术语、资料与署名

D.1 常用术语

术语	含义
非交互模式	接收一次任务后运行并输出，不进入持续全屏对话
上下文	Agent 当前能看到的指令、历史、代码与工具结果
基线	修改前的文件、测试和环境事实，用于比较变化
证据	日志、截图、下载文件、测量数据等可复核材料
定向复验	针对刚修的问题重跑指定场景，并检查受影响功能
mock	本地模拟服务，不代表真实上游服务
worktree	Git 的独立工作树，用于隔离修改；不保护未跟踪文件

D.2 资料来源

- Kimi Code 官方文档：Agents and Sub-Agents，用于核对主/子 Agent、上下文隔离、权限与全局指令位置。
- Kimi Code 官方文档：Built-in Tools，用于核对 Bash 后台任务、通知、TaskOutput/TaskStop 和 TodoList。
- Kimi Code 官方文档：Getting Started，用于核对 `kimi -p` 与基本使用方式。
- 本机 `zcode 0.17.0 --help`、`codex-cli 0.162.1 --help`、`codex exec --help`。参数具有版本性，使用时应重新核对。
- 本教程示例均为虚构，不引用实际项目材料，不证明任何真实任务已完成或通过验收。

D.3 作者与署名

作者：simon，一个不懂代码的AI极客
微信交流群：simonsxxs
GitHub：https://github.com/simonsxxs
Gitee：https://gitee.com/simonsxx

先在一个安全的演示任务中跑通“任务书 → 执行记录 → 证据 → 独立审查”，再把稳定规则写入全局 `AGENTS.md`。这样既能减少重复操作，也能保留必要的权限控制和质量检查。